

Langchain Runnables - Part 2

1. Introduction

In Part 1, you learned:

- What Runnables are
- How they standardize LangChain components

👉 In Part 2, we go deeper into:

- Different **types of Runnables**
 - How to use them to build **real workflows**
-

2. Recap

Quick reminder:

👉 Runnable = a **building block** that:

- Takes input
- Processes it
- Returns output

And supports:

- `.invoke()` → single input
 - `.batch()` → multiple inputs
 - `.stream()` → streaming output
-

3. Runnable Categories

Runnables are not just one thing, they come in different **types**, each solving a different problem:

✅ **Main Categories:**

1. **RunnableSequence** → step-by-step flow
 2. **RunnableParallel** → run tasks together
 3. **RunnablePassthrough** → pass data forward
 4. **RunnableLambda** → custom logic
 5. **RunnableBranch** → conditional logic
-

👉 Think of these like tools to build **any workflow you want**

4. RunnableSequence

✅ **What it is:**

A **sequence of steps** where each step runs one after another

💡 **Same as:**

👉 Sequential Chain (but cleaner)

Example:

chain = prompt | model | parser

Flow:

Input → Prompt → Model → Parser → Output

✅ **Use Case:**

- Blog generation
 - Data processing pipeline
 - Step-by-step tasks
-

💡 **Key Idea:**

Each step depends on the previous output

5. RunnableParallel

✅ **What it is:**

Runs **multiple tasks at the same time**

Example:

```
from langchain_core.runnables import RunnableParallel

chain = RunnableParallel({
    "summary": summary_chain,
    "keywords": keyword_chain
})
```

Flow:

→ Summary

Input → → Keywords → Combined Output

✅ **Use Case:**

- Generate multiple outputs
-

- Save time by parallel execution

Key Idea:

Tasks are **independent**

6. RunnablePassthrough

What it is:

Passes input **unchanged** to the next step

Why needed?

Sometimes you want:

- Original input + processed output together
-

Example:

```
from langchain_core.runnables import RunnablePassthrough

chain = RunnablePassthrough() | model
```

Better Example (Important):

```
chain = {
    "original": RunnablePassthrough(),
    "processed": model
}
```

Output:

```
{
  "original": "Hello",
  "processed": "Hi there!"
}
```

Use Case:

- Keep original data
 - Combine multiple outputs
-

7. RunnableLambda

What it is:

Allows you to use a **custom Python function** inside a chain

 Think:

 “I want to add my own logic here”

Example:

```
from langchain_core.runnables import RunnableLambda

def to_upper(text):
    return text.upper()

chain = RunnableLambda(to_upper)
```

Output:

```
Input:
hello
Output:
HELLO
```

 Use Case:

- Data transformation
 - Custom logic
 - Pre/post-processing
-

 Very Important:

 This connects **your code + LLM pipeline**

8. RunnableBranch

 What it is:

Used for **conditional logic (if-else)**

 Think:

 “If condition → do this

 Else → do that”

Example:

```
from langchain_core.runnables import RunnableBranch
```

```
chain = RunnableBranch(
    (lambda x: "translate" in x, translator_chain),
    (lambda x: "summarize" in x, summary_chain),
    default_chain
)
```

Flow:

Input → Check Condition → Choose Chain → Output

✅ Use Case:

- Smart assistants
 - Dynamic workflows
 - Multi-purpose apps
-

9. LCEL (LangChain Expression Language)

✅ What is LCEL?

LCEL is:

A simple way to build runnable pipelines using operators like |

💡 Example:

chain = prompt | model | parser

👉 This is LCEL in action

✅ Why LCEL is powerful:

1. Clean syntax
 2. Easy to read
 3. Easy to build pipelines
 4. Replaces complex chain code
-

Without LCEL:

❌ Complex and verbose

With LCEL:

✅ Simple and readable

🔥 Putting It All Together

Complete Workflow Example

Task: Process user input

Pipeline:

```
chain = (  
    RunnablePassthrough()  
    | RunnableLambda(clean_text)  
    | model  
    | parser  
)
```

Advanced Example:

```
chain = RunnableBranch(  
    (is_question, qa_chain),  
    (is_summary, summary_chain),  
    default_chain  
)
```

Important Concepts to Remember

1. **RunnableSequence** = step-by-step
 2. **RunnableParallel** = run tasks together
 3. **RunnablePassthrough** = keep original input
 4. **RunnableLambda** = add custom logic
 5. **RunnableBranch** = conditional flow
 6. **LCEL** = clean way to connect everything
-

Big Picture

Before:

-  Many separate tools
-  Complex code

After Runnables + LCEL:

-  Everything unified
 -  Clean pipelines
 -  Scalable systems
-

✅ Final Summary

- Runnables are the **core building blocks**
 - Different types help solve different problems:
 - Sequence → order
 - Parallel → speed
 - Passthrough → data flow
 - Lambda → custom logic
 - Branch → decisions
 - LCEL makes everything **simple and readable**
-